Imputation of PEG methylation data to PPMI reference panel

Under **../rawdata/**, save the following files into **impute_cpg/** directory:

Fetch the PPMI betas file that is generated from the ppmi_pipeline from the PPMI Box folder and save this as **data_ppmi.txt** in **impute_cpg/**.

Fetch the PEG beta files from the PEG Box folder and save these as **data_peg1.txt** and **data_peg2.txt**.

Save the manifest files from the PPMI Box folder by downloading **manifest.tar.gz** and decompressing this file.

Next we will divide up the large datamatrices into Chromosome sized chunks. Run the following in Python:

```
import sys
import os
os.chdir('../rawdata/impute_cpg')

filename_450k='coordinates_450k.csv'
filename_epic1='coordinates_epic_v1.csv'
filename_epic2='coordinates_epic_v2.csv'
filename_samples_peg1='peg1_samples.txt'
filename_samples_peg2='peg2_samples.txt'
filename_data_ppmi='data_ppmi.txt'
filename_data_peg1='data_peg1.txt'
filename_data_peg2='data_peg2.txt'
cpg2chr={}
pheno_listnames<-c('ppmi','peg1','pegall')
peg1_samples=[]
peg2_samples=[]
ppmi_samples=[]
with open(filename_samples_peg1,'r') as file:
    for line in file:
        peg1_samples.append(line.strip())
with open(filename_samples_peg2,'r') as file:
    for line in file:
        peg2_samples.append(line.strip())
with open(filename_data_ppmi,'r') as file:
    ppmi_samples=file.readline().strip().split('\t')



with open(filename_450k,'r') as file:
    file.readline()
    for line in file:
        if 'Controls' in line:
            break
        cpg,chrom,_,_,_=line.strip().split(',')
        if chrom=="\\N":
            chrom="NA"
        cpg2chr[cpg] = chrom
```

```
print(f"Length {len(cpg2chr)}",file=sys.stderr)
with open(filename_epic1,'r') as file:
    file.readline()
    for line in file:
        if 'Controls' in line:
            break
        tokens = line.strip().split(',')
        _,cpg,chrom,_=line.strip().split(',')
        if chrom=="\\N":
            chrom="NA"
        cpg2chr[cpg] = chrom
print(f"Length {len(cpg2chr)}",file=sys.stderr)
with open(filename_epic2,'r') as file:
    file.readline()
    for line in file:
        if 'Controls' in line:
            break
        _,cpg,chrom,_=line.strip().split(',')
        if chrom=="\\N":
            chrom="NA"
        cpg2chr[cpg] = chrom
print(f"Length {len(cpg2chr)}",file=sys.stderr)

peg1in = open(filename_data_peg1,'r')
peg2in = open(filename_data_peg2,'r')
ppmiin = open(filename_data_ppmi,'r')

outdir='chunks/'
try:
  os.mkdir(outdir)
except FileExistsError:
  print(f"{outdir} exists already.")
def split_input_ppmi(filein,prefix):
    filein.readline()
    files_by_chr={}
    i=0
    for line in filein:
        tokens=line.strip().split('\t')
        cpg=tokens[0]
        vals=tokens[1:]
        chrom=cpg2chr[cpg]
        #print(f'Chrom {chrom}')
        if chrom not in files_by_chr:
            fileout = open(prefix+'_chr_'+chrom+'.txt','w')
            fileout.write('\t'.join(ppmi_samples)+'\n')
            files_by_chr[chrom] = fileout
        else:
            fileout = files_by_chr[chrom]
        fileout.write("\t".join(tokens)+"\n")
        if i%1000==0:
            print(f"{i} lines written")
        i+=1
    for fileout in files_by_chr.values():
```

```
        fileout.close()
    filein.close()

def split_input_peg(sampleslist,filein,prefix):
    print(f"Running {prefix}")
    files_by_chr={}
    i=0
    for line in filein:
        tokens=line.strip().split('\t')
        cpg=tokens[0]
        valstring = tokens[1]
        vals=valstring.split(',')
        chrom=cpg2chr[cpg]
        #print(f'Chrom {chrom}')
        if chrom not in files_by_chr:
            fileout = open(prefix+'_chr_'+chrom+'.txt','w')
            fileout.write('\t'.join(sampleslist)+'\n')
            files_by_chr[chrom] = fileout
        else:
            fileout = files_by_chr[chrom]
        fileout.write(cpg+"\t"+"\t".join(vals)+"\n")
        if i%1000==0:
            print(f"{i} lines written")
        i+=1
    for fileout in files_by_chr.values():
        fileout.close()
    filein.close()

#split_input_ppmi(ppmiin,outdir+'ppmi')
#split_input_peg(peg1_samples,peg1in,outdir+'peg1')
split_input_peg(peg2_samples,peg2in,outdir+'peg2')
```

The output of the code above will be 22 **.rds** files corresponding to the chunked files where rows are CPGs and columns are subjects concatenated as PPMI,PEG1, and PEG2.

The following R code will perform a SVD-based imputation using information from the PPMI EPIC methylation panel as a reference panel. It will generate density plots of the Betas for each of the cohorts, as well as QQ plots of the distribution of p-values in a differential methylation probe analysis using disease status as the outcome. These plots are intended as a sensitivity analysis for the hyperparameters of the low rank approximation method. The code can be run in parallel on the Hoffman cluster or locally.

Running locally

```
task_id=as.numeric(Sys.getenv('SGE_TASK_ID'))

# determine if we're on a cluster
is_local<-is.na(task_id)

if (!is_local){
  task_df<-read.table('tasks.txt',header=T)
  selected_chr<-task_df[task_df$task_id==task_id,"chrom"]
  selected_lambda<-task_df[task_df$task_id==task_id,"lambda"]
  selected_rank<-task_df[task_df$task_id==task_id,"rank"]
```

```
}
ppmi_samples<-1983
peg1_samples<-580
peg2_samples<-227
ppmi_indices<-seq(1,ppmi_samples)
offset<-max(ppmi_indices)
peg1_indices<-seq(offset+1,offset+peg1_samples)
offset<-max(peg1_indices)
peg2_indices<-seq(offset+1,offset+peg2_samples)

chroms<-as.character(seq(1:22))
lambdas<-c(50,40,30,20)
ranks=c(15,10,5)
# ultimately when we want to output the imputed values we set these values
optimal_lambda<-50
optimal_rank<-15

library(sva)
library(softImpute)
#library(methyLImp2)
library(BiocParallel)
library(SummarizedExperiment)
# for KNN
library(impute)
library(ChAMP)
library(ggplot2)
#setwd('~/analysis/PD-QTLs/rawdata/impute_cpg/')
get_disease_code<-function(val){
  if (val=='Parkinsons.Disease.BL'){
    return(1)
  }else if (val=='Healthy.Control.BL'){
    return(0)
  }
  return(NA)
}
get_slide<-function(val){
  left_stripped<-substring(val,2)
  tokens<-strsplit(left_stripped,'_')
  return(tokens[[1]] [1])
}
get_array<-function(val){
  left_stripped<-substring(val,2)
  tokens<-strsplit(left_stripped,'_')
  return(tokens[[1]][2])
}


plot_column<-function(column,name){
  dat<-data.frame(betas=c(column),
                  lines = c(rep(name,length(column))))

  p<-ggplot(dat,aes(x=betas,colour=lines)) + geom_density(alpha=0.5)
  ggsave(paste('plots/',name,'_density.png',sep=''),plot=p)
```

```
#  ggplot(dat, aes(sample = betas))
#         p + stat_qq() + stat_qq_line()

}

plot_densities<-function(df_epic_450k,completed,plotdir){
  completed_ppmi<-completed[,12:1983]
  annotated_completed_ppmi<-merge(df_epic_450k,completed_ppmi,by.x='epic_ID',by.y=0)
  typed_ppmi<-annotated_completed_ppmi[!is.na(annotated_completed_ppmi$CHR.y),]
  imputed_ppmi<-annotated_completed_ppmi[is.na(annotated_completed_ppmi$CHR.y),]

  typed_ppmi_v <- stack(imputed_ppmi[,12:1983])$values
  # plot_column(typed_ppmi_v,paste(name,'typed_ppmi',sep='_'))
  imputed_ppmi_v <-  stack(typed_ppmi[,12:1983])$values
  # plot_column(imputed_ppmi_v,paste(name,'imputed_ppmi',sep='_'))

  #plot(density())
  #plot(density())

  peg1_samples<-580
  peg2_samples<-227
  total_samples<-peg1_samples+peg2_samples

  completed_peg<-completed[,1984:(1984+total_samples-1)]
  annotated_completed_peg<-merge(df_epic_450k,completed_peg,by.x='epic_ID',by.y=0)
  typed_peg<-annotated_completed_peg[!is.na(annotated_completed_peg$CHR.y),]
  imputed_peg<-annotated_completed_peg[is.na(annotated_completed_peg$CHR.y),]

  peg1_offset=12
  peg2_offset=peg1_offset+580
  peg1_end = peg2_offset-1
  peg2_end = peg2_offset+peg2_samples-1

  typed_peg1_v=stack(typed_peg[,peg1_offset:peg1_end])$values
  # plot_column(typed_peg1_v,paste(name,'typed_peg1',sep='_'))
  imputed_peg1_v = stack(imputed_peg[,peg1_offset:peg1_end])$values
  # plot_column(imputed_peg1_v,paste(name,'imputed_peg1',sep='_'))

  typed_peg2_v=stack(typed_peg[,peg2_offset:peg2_end])$values
  # plot_column(typed_peg2_v,paste(name,'typed_peg2',sep='_'))
  imputed_peg2_v = stack(imputed_peg[,peg2_offset:peg2_end])$values
  # plot_column(imputed_peg2_v,paste(name,'imputed_peg2',sep='_'))


  dat<-data.frame(betas=c(typed_peg1_v,imputed_peg1_v,typed_peg2_v,imputed_peg2_v,typed_ppmi_v,imputed_
                lines = c(rep('typed_peg1',length(typed_peg1_v)),
                          rep('imputed_peg1',length(imputed_peg1_v)),
                          rep('typed_peg2',length(typed_peg2_v)),
                          rep('imputed_peg2',length(imputed_peg2_v)),
                          rep('typed_ppmi',length(typed_ppmi_v)),
                          rep('imputed_ppmi',length(imputed_ppmi_v))
                ))
```

```
  p<-ggplot(dat,aes(x=betas,colour=lines)) + geom_density(alpha=0.5)


  ggsave(paste(plotdir,'/beta_densities.png',sep=''),plot=p)
  #  ggplot(dat_ppmi,aes(x=betas,colour=lines)) + geom_density(alpha=0.5)
  #ggsave('ppmi_density.png')

}

# methyLImp2_impute<-function(df){
#   beta_SE_imputed <- methyLImp2(input = t(as.matrix(df)),
#                                 type = "EPIC",
#                                 BPPARAM = SnowParam(exportglobals = FALSE),
#                                 minibatch_frac = 0.5)
#   return (t(beta_SE_imputed))
# }

svd_impute<-function(df,rank,lambda){
  df_c=biScale(df,col.scale=FALSE,row.scale=FALSE,trace=TRUE)
  print(paste('svd_impute: lambda',lambda,'rank',rank,sep=' '))
  fits3=softImpute(df_c,rank.max=rank,lambda=lambda,type="svd")
  print('Singular values:')
  print(fits3$d)
  completed<-complete(df,fits3,unscale=TRUE)

  return (completed)
}

knn_impute<-function(df){
  # now try KNN imputation
  knn_imputed<-impute.knn(as.matrix(merged))
  return(knn_imputed)
}


#from https://slowkow.com/notes/ggplot2-qqplot/

gg_qqplot <- function(ps, ci = 0.95,plotdir,label) {
  n  <- length(ps)
  df <- data.frame(
    observed = -log10(sort(ps)),
    expected = -log10(ppoints(n)),
    clower   = -log10(qbeta(p = (1 - ci) / 2, shape1 = 1:n, shape2 = n:1)),
    cupper   = -log10(qbeta(p = (1 + ci) / 2, shape1 = 1:n, shape2 = n:1))
  )
  log10Pe <- expression(paste("Expected -log"[10], plain(P)))
  log10Po <- expression(paste("Observed -log"[10], plain(P)))
  p<-ggplot(df) +
    geom_ribbon(
      mapping = aes(x = expected, ymin = clower, ymax = cupper),
      alpha = 0.1
    ) +
    geom_point(aes(expected, observed), shape = 1, size = 3) +
```

```
    geom_abline(intercept = 0, slope = 1, alpha = 0.5) +
    # geom_line(aes(expected, cupper), linetype = 2, size = 0.5) +
    # geom_line(aes(expected, clower), linetype = 2, size = 0.5) +
    xlab(log10Pe) +
    ylab(log10Po)
  ggsave(paste(plotdir,'/',label,'.png',sep=''),plot=p)
}


get_pheno_data<-function(){
  #read from file
  ppmi_pheno<-read.table('phenotypes_ppmi.txt',header=T)
  peg1_pheno<-read.table('peg1_pd.csv',sep=',',header=T)
  peg2_pheno<-read.table('peg2_pd.csv',sep=',',header=T)
  #factorize disease col
  ppmi_pheno$disease<-as.factor(unlist(lapply(ppmi_pheno$Sample_Group,get_disease_code)))
  ppmi_pheno$cohort<-as.factor('ppmi')
  peg1_pheno$PDstudyDiseaseNumeric<-as.factor(peg1_pheno$PDstudyDiseaseNumeric)
  peg1_pheno$cohort<-as.factor('peg1')
  peg2_pheno$disease<-as.factor(1)
  peg2_pheno$cohort<-as.factor('peg2')
  #split into slide and array for peg2
  peg2_pheno$Slide<-unlist(lapply(peg2_pheno$Sample_Name,get_slide))
  peg2_pheno$Array<-unlist(lapply(peg2_pheno$Sample_Name,get_array))
  #standardize col names
  standard_names<-c('Sample_Name','Slide','Array','Sample_Group','cohort')
  ppmi_reduced<-ppmi_pheno[,c('Sample_Name','Slide','Array','disease','cohort')]
  colnames(ppmi_reduced)<-standard_names
  peg1_reduced<-peg1_pheno[,c('ExternalDNACode','Slide','Array','PDstudyDiseaseNumeric','cohort')]
  colnames(peg1_reduced)<-standard_names
  peg2_reduced<-peg2_pheno[,c('PEGID','Slide','Array','disease','cohort')]
  colnames(peg2_reduced)<-standard_names
  pegall_reduced<-rbind(peg1_reduced,peg2_reduced)
  return(list(ppmi_reduced,peg1_reduced,pegall_reduced))
  #colnames(all_reduced)<-c('Sample_Name','Slide','Array','Sample_Group')
  #all_reduced$Sample_Group<-as.factor(all_reduced$Sample_Group)
}
get_epic_450k<-function(chrom){
  # read annotations
  df_epicv1<-read.csv('coordinates_epic_v1.csv')
  df_epicv1<-df_epicv1[df_epicv1$CHR==chrom,]
  df_epicv2<-read.csv('coordinates_epic_v2.csv')
  df_epicv2<-df_epicv2[df_epicv2$CHR_37==chrom,]
  df_epic<-merge(df_epicv1,df_epicv2,all=TRUE,by='Name')
  colnames(df_epic)[1] = 'epic_ID'
  colnames(df_epic)[2] = 'epic1_ID'
  colnames(df_epic)[5] = 'epic2_ID'
  df_450k<-read.csv('coordinates_450k.csv')
  colnames(df_450k)[1] = '450k_ID'
  df_450k<-df_450k[df_450k$CHR==chrom,]
  df_epic_450k<-merge(df_epic,df_450k,all=TRUE,by.x='epic_ID',by.y='450k_ID')

  return(df_epic_450k)
```

```
}

get_450k<-function(chrom){
  # read annotations

  df_450k<-read.csv('coordinates_450k.csv')
  colnames(df_450k)[1] = '450k_ID'
  df_450k<-df_450k[df_450k$CHR==chrom,]
  return(df_450k)
}

get_cpg_lists<-function(df_epic_450k,df_data){

  annotated<-merge(df_epic_450k,df_data,by.x='epic_ID',by.y=0)
  typed<-annotated[!is.na(annotated$CHR.y),]
  imputed<-annotated[is.na(annotated$CHR.y),]
  return(list(typed$epic_ID,imputed$epic_ID))
}

#pheno_df<-pheno_list[[1]]
#cpg_list<-cpg_lists[[1]]

dmp_subset<-function(corrected,pheno_df,cpg_list){
  sample_names<-pheno_df$Sample_Name
  pheno_df2<-pheno_df[!is.na(pheno_df$Sample_Group),]

  completed_subset<-corrected[cpg_list,sample_names]
  completed_subset2<-completed_subset[,!is.na(pheno_df$Sample_Group)]

  myDMP<-champ.DMP(beta=completed_subset2,pheno=pheno_df2$Sample_Group,compare.group = c(0,1),adjPVal =
  return (myDMP)
}

batch_correct<-function(df_data,df_pheno,plotdir,label){

  df_pheno$cohort_slide<-as.factor(paste(df_pheno$cohort,df_pheno$Slide,sep='_'))
  modcombat = model.matrix(~1, data=df_pheno)
  corrected<-ComBat(dat=df_data,batch=df_pheno$cohort_slide,mod=modcombat,par.prior=TRUE)
  #corrected<-champ.runCombat(beta=df_data,pd=df_pheno,variablename='Sample_Group',batchname=c('Slide',

  # begin overdisperson diagnostics on case vs controls
  corrected2<-corrected[,!is.na(df_pheno$Sample_Group)]
  df_pheno2<-df_pheno[!is.na(df_pheno$Sample_Group),]
  mod=model.matrix(~as.factor(Sample_Group),data=df_pheno2)
  mod0=model.matrix(~1,data=df_pheno2)
  pValuesComBat = f.pvalue(corrected2,mod,mod0)
  gg_qqplot(ps=pValuesComBat,plotdir=plotdir,label='combat_pd')
  mod2=model.matrix(~as.factor(sample(Sample_Group)),data=df_pheno2)
  pValuesComBatNull = f.pvalue(corrected2,mod2,mod0)
  gg_qqplot(ps=pValuesComBatNull,plotdir=plotdir,label='combat_null')
  return(as.data.frame(corrected))
}
```

```r
filter_cpgs<-function(myImport,arraytype,pheno_list){
  myFiltered<-champ.filter(beta=myImport$beta,SampleCutoff = 1.0,arraytype=arraytype,fixOutlier = FALSE)
    # assign sample names to columns
  newdf<-as.data.frame(myFiltered$beta)
  colnames(newdf)<-c(pheno_list[[1]]$Sample_Name,pheno_list[[3]]$Sample_Name)
  return(newdf)
}

blank_row<-function(val){
  if(!is.na(val) && runif(1)>.5) {
    val = NA
  }
  return(val)
}

add_blanks<-function(df){
  ppmi_subset<-df[,ppmi_indices]
  ppmi_defined<-!is.na(apply(ppmi_subset,1,sum))
  peg_subset_to_blank<-df[ppmi_defined,c(peg1_indices,peg2_indices)]
  peg_subset_no_blank<-df[!ppmi_defined,c(peg1_indices,peg2_indices)]
  blanked<-apply(peg_subset_to_blank,c(1,2),blank_row)
  peg_subset<-rbind(blanked,peg_subset_no_blank)
  peg_subset<-peg_subset[rownames(ppmi_subset),]
  newdf<-cbind(ppmi_subset,peg_subset)
  return(newdf)
}


get_deltas<-function(is_imputed,truth,imputed){
  delta<-NA
  if (is_imputed==TRUE){
    delta<-(imputed-truth)
  }
  return(delta)
}

compute_r_squared<-function(){
  for (chrom in chroms){
    if (is_local || chrom==selected_chr){
      myImport<-list('beta')
      print(paste("Chromosome",chrom,sep=' '))
      infilename<-paste('chunks/merged_',chrom,'.rds',sep='')
      # load from file
      raw<-readRDS(infilename)
      df_450k<-get_450k(chrom)
      myImport$beta<-raw[df_450k$`450k_ID`,]
      merged_450k<-filter_cpgs(myImport,'450K',pheno_list)
      isdefined<-!is.na(apply(merged_450k,1,sum))
      merged_450k<-merged_450k[isdefined,]
      blanked_450k<-add_blanks(merged_450k)
      cpgmatrix_is_blanked<-!is.na(merged_450k) & is.na(blanked_450k)
      for (rank in ranks){
        for (lambda in lambdas){
```

```
          if (is_local || (rank==selected_rank && lambda==selected_lambda)){
            label1<-paste('chrom',chrom,'rank',rank,'lambda',lambda,'',sep='_')
            plotdir<-paste('plots',label1,sep='/')
            if (!dir.exists(plotdir)){
              dir.create(plotdir)
            }
            print(paste("Label",label1,sep=" "))
            completed_450k<-svd_impute(blanked_450k,rank,lambda)
            delta_matrix<-cpgmatrix_is_blanked * (merged_450k - completed_450k) *(merged_450k - complete
            total_blanked<-sum(cpgmatrix_is_blanked)
            sse<-sum(delta_matrix)/total_blanked
            mean<-sum(cpgmatrix_is_blanked * merged_450k)/total_blanked
            sst<-sum(cpgmatrix_is_blanked * (merged_450k-mean)*(merged_450k-mean))
            rsquared<- 1-(sse/sst)
            label1<-paste(label1,rsquared,sep=':')
            cat('\n')
            cat(label1)
            cat('\n')
          }
        }
      }
    }
  }
}

impute<-function(){
  for (chrom in chroms){
    if (is_local || chrom==selected_chr){
      myImport<-list('beta')
      print(paste("Chromosome",chrom,sep=' '))
      infilename<-paste('chunks/merged_',chrom,'.rds',sep='')
      # load from file
      raw<-readRDS(infilename)
      df_epic_450k<-get_epic_450k(chrom)
      myImport$beta<-raw[df_epic_450k$epic_ID,]
      # filter out nonCPGs using CHAMP
      merged_all<-filter_cpgs(myImport,'EPIC',pheno_list)
      #cpgmatrix_is_blanked_peg<-cpgmatrix_is_blanked[,c(peg1_indices,peg2_indices)]
      # get the join of epic and 450k
      cpg_lists<-get_cpg_lists(df_epic_450k,merged_all)
      cpg_listnames<-c('typed','imputed')
      for (rank in ranks){
        for (lambda in lambdas){
          if (is_local || (rank==selected_rank && lambda==selected_lambda)){
            label1<-paste('chrom',chrom,'rank',rank,'lambda',lambda,'',sep='_')
            plotdir<-paste('plots',label1,sep='/')
            if (!dir.exists(plotdir)){
              dir.create(plotdir)
            }
            print(paste("Label",label1,sep=" "))
            completed<-svd_impute(merged_all,rank,lambda)
            plot_densities(df_epic_450k,completed,plotdir)
            all_pheno<-rbind(pheno_list[[1]],pheno_list[[3]])
```

```
            corrected<-batch_correct(completed,all_pheno,plotdir,label1)
            if (rank==optimal_rank && lambda==optimal_lambda){
              outfilename=paste('imputed/corrected_',chrom,'.rds',sep='')
              saveRDS(corrected,file=outfilename)
            }
            for (i in 1:length(pheno_listnames)){
              for (j in 1:length(cpg_listnames)){
                print(paste('pheno',pheno_listnames[i],'cpglist',cpg_listnames[j],sep=' '))
                myDMP<-dmp_subset(corrected,pheno_list[[i]], cpg_lists[[j]])
                subset_name<-paste(pheno_listnames[i],cpg_listnames[j],sep='_')
                gg_qqplot(myDMP[[1]]$P.Value,plotdir=plotdir,label=subset_name)
              }
            }
          }
        }
      }
    }
  }
}

pipeline<-function(){
  pheno_list<-get_pheno_data()
  compute_r_squared()
  impute()
}

pipeline()
```

Running on Hoffman cluster

To run on Hoffman, save the script above as cpg_impute.r. Then, generate a task list file:

```
setwd('../rawdata/impute_cpg/')

chroms<-as.character(seq(1:22))
lambdas<-c(50,40,30,20)
ranks=c(15,10,5)
i<-0
tasklist=list()

for(chrom in chroms){
  for(lambda in lambdas){
    for(rank in ranks){
      i<-i+1
      #chrom<-i
      #row<-list(c(i,rank,lambda))
      row<-list(c(i,chrom,rank,lambda))
      tasklist<-c(tasklist,row)
    }
  }
}
task_df<-t(do.call(data.frame,tasklist))
colnames(task_df)<-c('task_id','chrom','rank','lambda')
```

```
#colnames(task_df)<-c('task_id','rank','lambda')
write.table(task_df,file='tasks.txt',quote=F,row.names=F)
```

Edit the Hoffman array shell script file accordingly so that the line **-t 1-22:1** specifies the total tasks as shown in the **tasks.txt** file. In this example, there are 22 tasks (1-indexed). Save the file on Hoffman as array.sh

```
#### submit_arrayjob.sh START ####
#!/bin/bash
#$ -cwd
# error = Merged with joblog
#$ -o joblog.$JOB_ID.$TASK_ID
#$ -j y
## Edit the line below as needed:
#$ -l h_rt=2:30:00,h_data=32G
## Modify the parallel environment
## and the number of cores as needed:
#$ -pe shared 1
# Email address to notify
##  $ -M nobody
# Notify when
#$ -m bea
#$ -t 1-22:1

# echo job info on joblog:
echo "Job $JOB_ID.$SGE_TASK_ID started on:   " `hostname -s`
echo "Job $JOB_ID.$SGE_TASK_ID started on:   " `date `
echo " "

# load the job environment:
. /u/local/Modules/default/init/modules.sh
## Edit the line below as needed:
#module load gcc/4.9.3
module load R
module load gcc
module load cmake/3.30.0

## substitute the command to run your code
## in the two lines below:
#echo '/usr/bin/time -v ./add_by_component.sh'
#/usr/bin/time -v ./add_by_component.sh
#/usr/bin/ls
cd /u/home/g/garychen/cpg_impute
/usr/bin/time -v R --no-save < cpg_impute.r


# echo job info on joblog:
echo "Job $JOB_ID.$SGE_TASK_ID ended on:   " `hostname -s`
echo "Job $JOB_ID.$SGE_TASK_ID ended on:   " `date `
echo " "
```

Upload the rds files, **tasks.txt**, **array.sh**, and **cpg_impute.r** and to the Hoffman cluster. The job can be submitted as

```
qsub array.sh
```